

Windows PowerShell Language Quick Reference (Page 1)

Native Support for Different Type Systems

Windows PowerShell adapts WMI, XML, ASDI, ADO, and COM objects to provide a common syntax to access their properties and methods.

Example
\$g = Get-WmiObject Win32_Process
\$g[0].Name # instead of
\$g[0].Properties["Name"]

Arithmetic Binary Operators

+	Addition, concatenation
-	Subtraction
*	Multiplication, string repetition
/	Division
%	Modulus

Array Operations

Does this array have a 3 in it
1,2,3,5,3,2 –contains 3

Return all elements equal to 3:
1,2,3,5,3,2 –eq 3

Return all elements less than 3:
1,2,3,5,3,2 –lt 3

Test if 2 exists in collection:
if (1, 3, 5 –contains 2) ...
Other operators: -gt, -le, -ge, -ne

Arrays

“a”, “b”, “c”	array of strings
1,2,3	array of integers
@()	empty array
@(2)	array of one element
1,(2,3),4	array within array
“hi”	array of one element
\$a[5]	sixth element of array*
\$a[2][3]	fourth element of the third element
\$a[2..20]	Return elements 3 thru 21

* Arrays are zero based.

Assignment Operators

?, +=, -=, *=, /=, %=

Associative Arrays (Hashtables)

\$hash = @{}	Create empty hashtable
\$h=@{a=1;b=2}	Create and initialize a hashtable
\$hash.key1 = 1	Assign 1 to key “key1”
\$hash.key1	Returns value of key1
\$hash["key1"]	Returns value of key1

Boolean Values and Operators

TRUE	FALSE
\$TRUE	\$FALSE
Any string of length > 0 except the word “false”	Empty string or the string “false”
Any number !=0	Any number = 0
Array of length > 1	Array of length 0
Array of length 1 whose element is TRUE	Array of length 1 whose element is FALSE
A reference to any object	Null

Break (Scripting)

The break commands exits a loop. It can take an optional LABEL to break to

Example:
while (1)
{
 \$a = something
 if (\$a –eq 1) break;
}

Command Expansion Operators

\$()	Returns null
\$1,2,3	Returns an array containing 1,2,3.
\$(Get-Alias a*)	Returns evaluation of the expression
@(Get-Alias; Get-Process)	Executes the two commands and returns the results in an array

Comments

This is a comment because # is the first char of a token
\$a = “#This is not a comment...”
\$a = “something” # ...but this is.
Write-Host Hello#world

Comparison Operators

-eq	Equal
-ne	Not equal
-gt –ge	Greater than, greater than or equal to
-lt –le	Less than, less than or equal to

“i” or “c” may be prepended to get case-insensitive or case-sensitive operations (for example, -ceq)

Continue (Scripting)

The continue statement continues the next iteration of a loop without breaking out of it.
Example:
while (1)
{
 \$a = something
 if (\$a –eq 1) (continue)
 # This line is not reached unless \$a == 1
}
This line is never reached.

Dot Sourcing

Dot sourcing allows running functions, script blocks, and scripts in the current scope rather than a local one.

Example:
. MyFunction

If MyFunction sets a variable, it is set in the current scope rather than the function’s local scope.
\$a = { \$x = Get-Process | Select –First 2 }
. \$a #Evaluates the script block #in the current scope

Escape Sequences

The Windows PowerShell escape character is the backwards apostrophe, or ` . To make a character literal, precede it with ` . To specify a ` use ``.

Special escape sequences	
`0	(null)
`a	(alert)
`b	(backspace)
`f	(form feed)
`n	(new line)
`r	(carriage return)
`t	(tab)
`v	(vertical quote)

Execution Order

Windows PowerShell attempts to resolve commands in the following order: **aliases, functions, cmdlets, scripts, executables, and normal files.**

For (Scripting)

[.label] for ([initializer]; [condition]; [iterator]) {}

Example:
for (\$i = 0; \$i –lt 5; \$i++) {Write-Object \$i}

Foreach (Scripting)

:label]
foreach (identifier in collection) {}
Expression | foreach {}
Expression | foreach {BEGIN {} PROCESS {} END {}}
Examples:
\$i = 1,2,3
foreach (\$z in \$i) {Write-Object \$z}
Get-Process |foreach {BEGIN {\$x=1}
 PROCESS {\$X++}
 END {“\$X Processes”}}

Functions (Scripting)

function MyFunction {
 write-object \$args[0]
}

function test ([string]\$label=”default label”, [int]\$start=0)
{ BEGIN {\$x=\$start} PROCESS {“\$label: \$_”; \$x++}
 END {“\$x total”}
}

Filters (Scripting)

Filters are a shorthand way of writing a function with a PROCESS script block.
filter MyFilter {
 \$_name
}

If/elseif/else (Scripting)

if (condition) {...}
elseif (condition) {...}
else {...}

On the command line, the closing brace must be on the same line as elseif and else. This restriction does not apply to scripts

Invoke Operator

The & operator can be used to invoke a script block or the name of a command or function.

Example:
\$a = “Get-Process”
&\$a
\$a = { Get-Process | Select -First 2 }
&\$a

Logical Operators

!, -not, -and, -or

Method Calls

Methods can be called on objects. Examples:
\$a = “This is a string”
\$a.ToUpper()
\$a.SubString(0,3)
\$a.SubString(0,(\$a.length/2))
\$a.Substring((\$a.length/2), (\$a.length/3))

Static Methods are callable with the “::” operator
[DateTime]::IsLeapYear(2005)

Object Properties

An object’s properties can be referenced directly with the “.” operator.
\$a = Get-Date
\$a.Date
\$a.TimeOfDay.Hours
Static properties can be referenced with the “::” operator
[DateTime]::Now

Operator Precedence

In Windows PowerShell, operators are evaluated in the following precedence: () {}, @ \$, !, [,], ., &, ++ --, Unary + -, * / %, Binary + -, Comparison Operators, -and -or, |, > >>, =

Other Operators

,	Array constructor
..	Range operator
-contains	
-is	Type evaluator
-as	Type convertor
-band	Binary and
-bor	Binary or
-bnot	Binary not

Windows PowerShell Language Quick Reference (Page 2)

Return (Scripting)

The **return** command exits the current script or function and returns a value.

Example:
function foo {
 return 1
}

Scopes (Scripting)

Variables and other data elements may be instantiated in different scopes:

- Variables in the global scope are visible in all scopes.
- Variables in the script scope are visible to all scopes within that script file.
- Variables in the local scope are visible only in the current scope and its children.
- Private scope variables are visible only to that current scope.

A scope is created in the body of a shell function (see function creation)

Example:
\$global:a = 4
\$script:a = 5
\$local:a = 3
\$private:a = 6

Script Blocks

Commands and expressions can be stored in a script block object and executed later.

Example:

\$block = {Get-Process; \$a=1}
&\$block

Scripts

Windows PowerShell commands can be stored in and executed from script files. The file extension for Windows PowerShell scripts is “.ps1”. Parameters can be passed to a script and a script can return a value.

Example:
\$sum = MyAdder.ps1 1 2 3

Strings

String constants:
“this is a string, this \$variable is expanded as is \$(2+2)”

‘this is a string, this \$variable is not expanded’
@”
This is a “here string” which can contain anything including carriage returns and quotes. Expressions \$(2+2) are evaluated
”@
@’
“here string” with single quotes do not evaluate expressions. ‘@

String operators	
+	Concatenate two strings
*	Repeat a string some number of times
-f	Format a string (.NET format specifiers)
-replace	Replace operator "abcd" -replace "bc", "TEST" aTESTd
-match	Regular expression match
-like	Wildcard matching

Switch (Scripting)

The variable \$_ is available in the script. \$_ represents the current value being evaluated. If an array is used in switch, each element of the array is tested.

Example:
\$var = "word1","word2","word3"
switch -regex (\$var) {
 "word1" {"Multi-match Exact " + \$_ }
 "word2" {"Multi-match Exact " + \$_ }
 "w.*2" {"Pattern match Exact " + \$_ }
 default {"Multi-match Default " + \$_ }
}

Output:
Multi-match Exact word1
Multi-match Exact word2
Pattern match Exact word2
Multi-match Default word3

Throw

Throw provides the same functionality for scripts as the ThrowTerminatingError API does for cmdlets.

throw "Danger, Danger"
Danger, Danger
At line:1 char:6

+ throw <<<< "Danger, Danger"
Throw takes a string, exception, or ErrorRecord as an argument.

Traps

Trap [ExceptionType] {
 if (...)
 { continue
 # continue at the script statement after the one that cased the
 # trap; \$? is updated but no error record is generated
 } else (...)
 { Break
 # rethrow the exception
 }
}
Doing nothing will do what is specified in the
\$ErrorActionPreference setting

Type Operations (Scripting)

-is IS type (e.g. \$a -is [int])
-as convert to type (e.g. 1 -as [string] treats 1 as a string)

Until (Scripting)

do
{
...
} until (condition)

Variables

Format:
\$[scope:]name or \${anyname} or \${any path}

Examples:
\$a = 1
\${!@#%&*()}=3
\$global:a = 1 # Visible everywhere
\$local:a = 1 # defined in this scope and visible to children
\$private:a=1 # same as local but invisible to child scopes
\$script:a=1 # visible to everything in this script
\$env:path = "d:\windows"
\${C:\TEMP\testfile.txt}= "This writes to a file"
Get-Variable -scope 1 a #Gets value from the parent scope
Get-Variable -scope 2 a # grandparent

While (Scripting)

[[:label]] while (condition)
{
...
}

do
{
...
} while (condition)

Windows PowerShell Automatic Variables (Not Exhaustive)

\$\$	Last token of the previous command line
\$?	Boolean status of last command
\$^	First token of the previous command line
\$	Current pipeline object
\$Args	Arguments to a script or function
\$Error	Array of errors from previous commands
\$ForEach	Reference to the enumerator in a foreach loop
\$Home	The user's home directory; usually set to %HOMEDRIVE%\%HOMEPATH%
\$Host	Reference to the application hosting the POWERSHELL language
\$Input	Enumerator of objects piped to a script
\$LastExitCode	Exit code of last program or script
\$Matches	Hash table of matches found with the -match operator
\$PSHome	The installation location of Windows PowerShell
\$profile	The standard profile (may not be present)
\$StackTrace	Last exception caught by Windows PowerShell
\$Switch	Enumerator in a switch statement

Parsing

Windows PowerShell parses in two modes - command mode and expression mode. In expression mode, Windows PowerShell parses as most high-level languages parse: numbers are numbers; strings need to be quoted, and so on. Expressions are things such as:
2+2 → 4
"Hello" + " world" → Hello world
\$a = "hi"; \$a.length * 13 → 26
When parsing in command mode, strings do not need to be quoted and everything is

treated like a string except variables and things in parentheses. For example:
copy users.txt accounts.txt
users.txt and accounts.txt are treated as strings
write-host 2+2
2+2 is treated as string, not an expression to evaluate
copy \$src \$dest

\$src and \$dest are variables. Not needing to use quotation marks when working in a command shell can be very beneficial in the long run because it greatly reduces the amount of typing required.

The parsing mode is determined by the first token encountered. If the token is a number, variable, or quoted string, then the shell parses in expression mode. If the line starts with a letter, an & (ampersand), or a . (dot) followed by a space or a letter, the parsing is done in command mode.

2+2	Expression mode starts with a number.
"text"	Expression mode starts with a quotation mark.
text	Command mode starts with a letter.
& "text"	Command mode starts with an ampersand.
."f.ps1"	Command mode starts with a dot followed by a space.
.125	Expression mode starts with a dot, which is followed by a number—not a space or a letter.
.text	Command mode starts with a dot, which is part of the command name ".text"

It is very useful to be able to mix expressions and commands; which you can do by using parentheses. Inside parentheses, the mode discovery process starts over.

Write-Host (2+2)
2+2 is treated as an expression to evaluate and is passed to the **Write-Host** command.
(Get-Date).day + 2

Get-Date is treated as a command, and the result of executing it becomes the left value in the expression.

You can nest commands and expressions without restrictions.
Write-Host ((Get-Date).day + 2)

Write-Host (((Get-Date) - (Get-Date).date)
The **Get-Date** command is used twice to determine how much time has passed since midnight (condition).